

Skill17.18.19.20

17. To Implement how to List Active Jobs, Display Status Information, Retrieve Job IDs, Format Output, Update State Information, Test Job Listings.

To design how to Identify Target Job, Transfer Terminal Control, Resume Processes, Wait for Completion, Update Job State, Test Foreground Switching.

INPUT:

```
GNU nano 8.7.1
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>

int main() {
    pid_t jobs[3];

    printf("Starting background jobs...\n");

    for (int i = 0; i < 3; i++) {
        jobs[i] = fork();

        if (jobs[i] == 0) {
            printf("Job %d running, PID = %d\n",
                i + 1, getpid());

            sleep(5);

            exit(0);
        }
    }

    printf("\nActive Jobs:\n");

    for (int i = 0; i < 3; i++) {
        printf("[%d] PID = %d - Running\n",
            i + 1, jobs[i]);
    }

    for (int i = 0; i < 3; i++)
        waitpid(jobs[i], NULL, 0);

    printf("\nAll jobs completed.\n");

    return 0;
}
```

OUTPUT:

```
sushmitha@LAPTOP-B907B06L:~$ nano skill17.c
sushmitha@LAPTOP-B907B06L:~$ gcc skill17.c -o skill17
sushmitha@LAPTOP-B907B06L:~$ ./skill17
Starting background jobs...
Job 1 running, PID = 600
Job 2 running, PID = 601

Active Jobs:
[1] PID = 600 - Running
Job 3 running, PID = 602
[2] PID = 601 - Running
[3] PID = 602 - Running

All jobs completed.
sushmitha@LAPTOP-B907B06L:~$ |
```

18.To Analyze Signal Delivery, Understand Signal Masks, Review Process Groups, Explore Terminal Signals, Test Signal Behavior.

To implement how to resume Stopped Jobs, Send Continue Signals, Update Job Status, Maintain Job Table, Verify Execution, Test Background Recovery.

INPUT:

```
GNU nano 8.7.1
#include <stdio.h>
#include <signal.h>
#include <unistd.h>

void handle_signal(int sig) {
    if (sig == SIGINT)
        printf("\nSIGINT received!\n");
}

int main() {
    signal(SIGINT, handle_signal);

    printf("Program running...\n");
    printf("Press Ctrl+C to send SIGINT.\n");

    while (1) {
        printf("Waiting for signal...\n");
        sleep(2);
    }

    return 0;
}|
```

OUTPUT:

```

sushmitha@LAPTOP-B907B06L:~$ nano skills.c
sushmitha@LAPTOP-B907B06L:~$ gcc skills.c -o skills
sushmitha@LAPTOP-B907B06L:~$ ./skills
Program running...
Press Ctrl+C to send SIGINT.
Waiting for signal...
Waiting for signal...
Waiting for signal...
Waiting for signal...
Waiting for signal...
Waiting for signal...
Waiting for signal...
Waiting for signal...
Waiting for signal...
Waiting for signal...
^C
SIGINT received!
Waiting for signal...
Waiting for signal...
^Z
[2]+  Stopped                               ./skills
sushmitha@LAPTOP-B907B06L:~$ |

```

19. To Register Handlers, Configure Signal Actions, Handle Interrupts, Manage Signal Context, Verify Stability, Test Signal Events.

To Capture SIGINT, Forward Signals, Protect Shell Process, Terminate Foreground Jobs, Update Job States, Test Interrupt Handling.

INPUT:

GNU nano 8.7.1

```
#include <stdio.h>
#include <signal.h>
#include <unistd.h>

void handle_signal(int sig) {
    if (sig == SIGINT)
        printf("\nSIGINT received!\n");
}

int main() {
    signal(SIGINT, handle_signal);

    printf("Program running...\n");
    printf("Press Ctrl+C to send SIGINT.\n");

    while (1) {
        printf("Waiting for signal...\n");
        sleep(2);
    }

    return 0;
}
```

OUTPUT:

```
sushmitha@LAPTOP-B907B06L:~$ nano skill19.c
sushmitha@LAPTOP-B907B06L:~$ gcc skill19.c -o skill19
sushmitha@LAPTOP-B907B06L:~$ ./skill19
Program running...
Press Ctrl+C to send SIGINT.
Waiting for signal...
Waiting for signal...
Waiting for signal...
Waiting for signal...
Waiting for signal...
Waiting for signal...
Waiting for signal...
Waiting for signal...
Waiting for signal...
Waiting for signal...
Waiting for signal...
Waiting for signal...
Waiting for signal...
Waiting for signal...
Waiting for signal...
^C
SIGINT received!
Waiting for signal...
Waiting for signal...
Waiting for signal...
Waiting for signal...
Waiting for signal...
Waiting for signal...
Waiting for signal...
^C
SIGINT received!
Waiting for signal...
^Z
[2]+  Stopped                  ./skill19
sushmitha@LAPTOP-B907B06L:~$ |
```

20.To Capture SIGTSTP, Suspend Processes, Update Job Table, Preserve State, Resume Later, Test Suspension Workflow.

To Create Process Groups, Assign Group IDs, Transfer Terminal Control, Restore Ownership, Coordinate Signals, Validate Behavior.

INPUT:

```
GNU nano 8.7.1
#include <stdio.h>
#include <signal.h>
#include <unistd.h>
#include <stdlib.h>

void sigint_handler(int sig) {
    printf("\nSIGINT caught by process.\n");
    printf("Cleaning resources...\n");
    printf("Program exiting safely.\n");

    exit(0);
}

int main() {
    signal(SIGINT, sigint_handler);

    printf("Process PID: %d\n", getpid());
    printf("Press Ctrl+C to terminate safely.\n");

    while (1) {
        printf("Running...\n");
        sleep(2);
    }

    return 0;
}
```

OUTPUT:

```
sushmitha@LAPTOP-B907B06L:~$ nano skill20.c
sushmitha@LAPTOP-B907B06L:~$ gcc skill20.c -o skill20
sushmitha@LAPTOP-B907B06L:~$ ./skill20
Process PID: 839
Press Ctrl+C to terminate safely.
Running...
Running...
Running...
Running...
Running...
Running...
Running...
Running...
Running...
Running...
^C
SIGINT caught by process.
Cleaning resources...
Program exiting safely.
sushmitha@LAPTOP-B907B06L:~$ |
```